```
# ============================================================================
# CASE PROJECT: BANKING INDUSTRY TRACK (Google Colab Setup)
# Dataset Path: /content/DsResearch.zip
# ============================================================================

# ----------------------------------------------------------------------------
# 1. IMPORT LIBRARIES & CONFIGURE ENVIRONMENT
# ----------------------------------------------------------------------------
import os
import warnings
import zipfile
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
    roc_curve,
)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams["figure.figsize"] = (10, 5)

# ----------------------------------------------------------------------------
# 2. EXTRACT ZIP & LOAD DATASET
# ----------------------------------------------------------------------------
ZIP_PATH = "/content/DsResearch.zip"
EXTRACT_DIR = "/content/DsResearch_Extracted"

print("Extracting zip archive...")
with zipfile.ZipFile(ZIP_PATH, "r") as zip_ref:
    zip_ref.extractall(EXTRACT_DIR)
print("Extraction complete!")

# Locate and read banking_data.csv
csv_path = None
for root, dirs, files in os.walk(EXTRACT_DIR):
    for file in files:
        if file.lower() == "banking_data.csv":
            csv_path = os.path.join(root, file)
            break

if not csv_path:
    raise FileNotFoundError(
        "Could not locate 'banking_data.csv' inside the extracted folder."
    )

print(f"Loading data from: {csv_path}")

# Load CSV (auto-detect delimiter)
try:
    df = pd.read_csv(csv_path, sep=";")
    if df.shape[1] == 1:
        df = pd.read_csv(csv_path, sep=",")
except Exception:
    df = pd.read_csv(csv_path)

print("\n" + "=" * 70)
print("DATASET OVERVIEW")
print("=" * 70)
print(f"Dimensions: {df.shape[0]} rows, {df.shape[1]} columns\n")
print(df.head())
```

```
print("\nColumn Information:")
print(df.info())

# ------------------------------------------------------------------------------
# 3. DATA PREPARATION & EDA
# ------------------------------------------------------------------------------
target_col = "y" if "y" in df.columns else "deposit"
print(f"\nTarget column identified: '{target_col}'")

# Convert target to binary (1 for yes, 0 for no)
df["target_binary"] = (
    df[target_col].apply(lambda x: 1 if str(x).strip().lower() in ["yes", "1"] else 0)
)

# 3.1 Target Distribution
plt.figure(figsize=(6, 4))
ax = sns.countplot(x=target_col, data=df, palette="viridis")
plt.title("Target Distribution (Subscription Outcome)")
plt.xlabel("Subscribed (yes/no)")
plt.ylabel("Count")
for p in ax.patches:
    ax.annotate(f"{p.get_height():,}", (p.get_x() + 0.3, p.get_height() + 100))
plt.tight_layout()
plt.show()

# 3.2 Conversion Rate by Job
if "job" in df.columns:
    plt.figure(figsize=(12, 4))
    job_conv = (
        df.groupby("job")["target_binary"].mean().sort_values(ascending=False) * 100
    )
    sns.barplot(x=job_conv.index, y=job_conv.values, palette="mako")
    plt.title("Conversion Rate (%) by Job Role")
    plt.xticks(rotation=45, ha="right")
    plt.ylabel("Conversion Rate (%)")
    plt.tight_layout()
    plt.show()

# 3.3 Conversion Rate by Education
if "education" in df.columns:
    plt.figure(figsize=(8, 4))
    edu_conv = (
        df.groupby("education")["target_binary"].mean().sort_values(ascending=False)
        * 100
    )
    sns.barplot(x=edu_conv.index, y=edu_conv.values, palette="crest")
    plt.title("Conversion Rate (%) by Education Level")
    plt.ylabel("Conversion Rate (%)")
    plt.tight_layout()
    plt.show()

# ------------------------------------------------------------------------------
# 4. PREPROCESSING & TRAIN-TEST SPLIT
# ------------------------------------------------------------------------------
df_model = df.drop(columns=["target_binary"], errors="ignore").copy()

cat_cols = df_model.select_dtypes(include=["object", "category"]).columns.tolist()
if target_col in cat_cols:
    cat_cols.remove(target_col)
num_cols = df_model.select_dtypes(include=[np.number]).columns.tolist()

y = df_model[target_col].apply(
    lambda x: 1 if str(x).strip().lower() in ["yes", "1"] else 0
)
X = df_model.drop(columns=[target_col])

# One-hot encode categorical features
X_encoded = pd.get_dummies(X, columns=cat_cols, drop_first=True)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, test_size=0.20, random_state=42, stratify=y
)

# Standard scaling
scaler = StandardScaler()
```

```
X_train_scaled = X_train.copy()
X_test_scaled = X_test.copy()

X_train_scaled[num_cols] = scaler.fit_transform(X_train[num_cols])
X_test_scaled[num_cols] = scaler.transform(X_test[num_cols])

# ------------------------------------------------------------------------------
# 5. MODEL TRAINING & EVALUATION
# ------------------------------------------------------------------------------
models = {
    "Logistic Regression": LogisticRegression(
        class_weight="balanced", max_iter=1000, random_state=42
    ),
    "Random Forest": RandomForestClassifier(
        n_estimators=200,
        class_weight="balanced",
        max_depth=12,
        random_state=42,
        n_jobs=-1,
    ),
    "Gradient Boosting": GradientBoostingClassifier(
        n_estimators=150, learning_rate=0.08, max_depth=5, random_state=42
    ),
}

results = []
trained_models = {}

plt.figure(figsize=(8, 6))

for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    trained_models[name] = model

    y_pred = model.predict(X_test_scaled)
    y_prob = model.predict_proba(X_test_scaled)[:, 1]

    results.append(
        {
            "Model": name,
            "Accuracy": accuracy_score(y_test, y_pred),
            "Precision": precision_score(y_test, y_pred),
            "Recall": recall_score(y_test, y_pred),
            "F1-Score": f1_score(y_test, y_pred),
            "ROC-AUC": roc_auc_score(y_test, y_prob),
        }
    )

    fpr, tpr, _ = roc_curve(y_test, y_prob)
    plt.plot(fpr, tpr, label=f"{name} (AUC = {roc_auc_score(y_test, y_prob):.3f})")

plt.plot([0, 1], [0, 1], "k--", label="Baseline")
plt.title("ROC-AUC Comparison Across Models")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.tight_layout()
plt.show()

# Print benchmark results
print("\n" + "=" * 70)
print("BENCHMARK METRICS SUMMARY")
print("=" * 70)
df_perf = pd.DataFrame(results).set_index("Model")
print(df_perf.round(4))

# ------------------------------------------------------------------------------
# 6. FEATURE IMPORTANCE (RANDOM FOREST)
# ------------------------------------------------------------------------------
rf_model = trained_models["Random Forest"]
feat_imp = pd.Series(
    rf_model.feature_importances_, index=X_train_scaled.columns
).sort_values(ascending=False)

plt.figure(figsize=(10, 6))
feat_imp.head(15).plot(kind="barh", color="teal")
```

```
plt.title("Top 15 Most Important Campaign Features (Random Forest)")
plt.xlabel("Importance Score")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```

```
Extracting zip archive...
Extraction complete!
Loading data from: /content/DsResearch_Extracted/DsResearch/Banking/banking_data.csv

======================================================================
DATASET OVERVIEW
======================================================================
Dimensions: 45216 rows, 19 columns

   age           job  marital marital_status  education default  balance  \
0   58    management  married        married   tertiary      no     2143
1   44    technician   single         single  secondary      no       29
2   33  entrepreneur  married        married  secondary      no        2
3   47   blue-collar  married        married    unknown      no     1506
4   33       unknown   single         single    unknown      no        1

  housing loan  contact  day month day_month  duration  campaign  pdays  \
0     yes   no  unknown    5   may     5-May       261         1     -1
1     yes   no  unknown    5   may     5-May       151         1     -1
2     yes  yes  unknown    5   may     5-May        76         1     -1
3     yes   no  unknown    5   may     5-May        92         1     -1
4      no   no  unknown    5   may     5-May       198         1     -1

   previous poutcome   y
0         0  unknown  no
1         0  unknown  no
2         0  unknown  no
3         0  unknown  no
4         0  unknown  no

Column Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45216 entries, 0 to 45215
Data columns (total 19 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   age             45216 non-null  int64
 1   job             45216 non-null  object
 2   marital         45213 non-null  object
 3   marital_status  45213 non-null  object
 4   education       45213 non-null  object
 5   default         45216 non-null  object
 6   balance         45216 non-null  int64
 7   housing         45216 non-null  object
 8   loan            45216 non-null  object
 9   contact         45216 non-null  object
 10  day             45216 non-null  int64
 11  month           45216 non-null  object
 12  day_month       45216 non-null  object
 13  duration        45216 non-null  int64
 14  campaign        45216 non-null  int64
 15  pdays           45216 non-null  int64
 16  previous        45216 non-null  int64
 17  poutcome        45216 non-null  object
 18  y               45216 non-null  object
dtypes: int64(7), object(12)
memory usage: 6.6+ MB
None

Target column identified: 'y'
```

